

In The Name of God



Ali Ahmadi (40464248)  
Komeil Khodayar (40464254)  
Navid Zare (40435071)

*Digital Image Processing*  
*Assignment 3: Hough Transform*

# 1. Introduction

The Hough Transform is one of the most fundamental techniques in computer vision for detecting parametric shapes in images. Originally proposed by Paul Hough in 1962 and later generalized by Duda and Hart (1972), the transform operates by mapping image-space features into a parameter space where geometric shapes manifest as peaks in an accumulator array. This approach is particularly robust to noise, partial occlusions, and gaps in edge contours, making it well-suited for real-world detection tasks.

This report presents the implementation and evaluation of a Hough Circle Transform applied to the problem of traffic light detection and state classification. Traffic light recognition is a critical component of autonomous driving systems and intelligent transportation infrastructure. The ability to accurately detect circular lamp regions within a traffic signal assembly and subsequently classify their illumination state (RED, GREEN, YELLOW, or UNLIT) represents a meaningful application of the Hough Circle Transform in a practical engineering context.

The primary objectives of this assignment are threefold.

- 1) First, a complete Canny edge detection pipeline is implemented from scratch, encompassing Gaussian smoothing, Sobel gradient computation, non-maximum suppression, double thresholding, and hysteresis-based edge linking.
- 2) Second, an optimized three-dimensional Hough Circle Transform is developed, exploiting gradient direction information to reduce computational complexity from the brute-force approach.
- 3) Third, a color-based classification module analyzes the interior pixels of each detected circle in HSV and RGB color spaces to determine the traffic light state.

All core algorithms are implemented strictly from scratch using NumPy vectorized operations, without reliance on built-in computer vision functions such as `cv2.Canny` or `cv2.HoughCircles`.

## 2. Methodology

### 2.1 Canny Edge Detection Pipeline

The edge detection module implements the full Canny algorithm, which consists of five sequential stages. Each stage is implemented from scratch using NumPy array operations to ensure both correctness and computational efficiency.

**Gaussian Smoothing.** The input grayscale image is first convolved with a two-dimensional Gaussian kernel to suppress high-frequency noise. The kernel is constructed as the outer product of a one-dimensional Gaussian vector with itself, exploiting the separability property of the Gaussian function. The kernel is defined as  $G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$ , where  $\sigma$  controls the degree of smoothing. A larger  $\sigma$  produces stronger smoothing, which reduces noise but may also blur fine edge details. The kernel size is set to  $5 \times 5$ , and  $\sigma$  is tuned per image (ranging from 1.2 to 1.6) to balance noise suppression with edge preservation.

**Sobel Gradient Computation.** The smoothed image is convolved with the standard  $3 \times 3$  Sobel kernels to compute the horizontal gradient  $G_x$  and vertical gradient  $G_y$ . The gradient magnitude is calculated as  $M = \sqrt{G_x^2 + G_y^2}$ , and the gradient direction is computed as  $\theta = \arctan\left(\frac{G_y}{G_x}\right)$ . The gradient direction is essential not only for the non-maximum suppression step but also for the subsequent Hough Circle Transform, where it guides the voting direction.

**Non-Maximum Suppression (NMS).** To produce thin, single-pixel-wide edges, the gradient magnitude at each pixel is compared against its two neighbors along the gradient direction. The gradient angle is quantized into four directional bins ( $0^\circ$ ,  $45^\circ$ ,  $90^\circ$ ,  $135^\circ$ ), and for each pixel, the magnitudes of the two neighboring pixels along the quantized direction are examined. A pixel is retained only if its magnitude is greater than or equal to both neighbors; otherwise, it is suppressed to zero. This entire operation is implemented using vectorized array slicing and conditional selection via *np.where*, avoiding explicit pixel-level iteration.

**Double Thresholding.** The NMS output is classified into strong and weak edges using two thresholds derived from the maximum magnitude value. Pixels with magnitude above the high threshold are classified as strong edges, while those between the low and high thresholds are classified as weak edges. All others are discarded. The threshold ratios are tuned per image, with typical values of 0.03–0.05 for the low threshold and 0.10–0.15 for the high threshold.

**Hysteresis Edge Linking.** Weak edge pixels are promoted to strong edges if they are connected (in an 8-neighborhood sense) to at least one strong edge pixel. This process is performed iteratively: in each iteration, weak pixels adjacent to any strong pixel are promoted, and the process repeats until no further promotions occur. The iterative propagation is implemented using padded Boolean mask arrays, ensuring that all connected weak edges along a contour are retained while isolated weak pixels are discarded.

## 2.2 Optimized Hough Circle Transform

The standard parametric equation for a circle is  $(x - a)^2 + (y - b)^2 = r^2$ , where  $(a, b)$  denotes the center coordinates and  $r$  denotes the radius. A brute-force Hough Circle Transform requires voting across a three-dimensional parameter space  $(a, b, r)$ , where for each edge pixel, votes are cast along a full  $360^\circ$  arc at every candidate radius. This results in computational complexity of  $O(E \times R \times 2\pi r)$ , which is prohibitively expensive for large images and wide radius ranges.

To address this, the implementation employs gradient direction optimization. For each edge pixel at position  $(x, y)$ , the gradient direction  $\theta$  computed during edge detection provides the normal direction to the contour at that point. Since the center of a circle lies along the gradient normal of each edge pixel, votes need only be cast at two candidate center positions per radius (along the positive and negative gradient directions), rather than along the full circumference. The candidate centers are computed as:  $a = x \pm r \cos(\theta)$  and  $b = y \pm r \sin(\theta)$ . This optimization reduces the per-pixel voting cost from  $O(2\pi r)$  to  $O(2)$ , yielding a dramatic overall speedup.

The accumulator is a three-dimensional array of shape  $(H, W, N_r)$ , where  $H$  and  $W$  are the image dimensions and  $N_r$  is the number of discrete radius bins. For each candidate radius, the batch of candidate centers is computed for all edge pixels simultaneously using vectorized NumPy operations. Votes are accumulated using `np.add.at`, which correctly handles unbuffered in-place addition even when multiple edge pixels vote for the same accumulator cell. This is necessary because standard indexed assignment (`accumulator[indices] += 1`) uses buffered semantics and would count duplicate indices only once.

**Peak Detection.** After the voting phase, peaks in the accumulator are identified using a local maximum criterion. A global vote threshold is set as a fraction (configurable per image, typically 0.35–0.50) of the maximum accumulator value. For each radius slice, only pixels whose vote count exceeds this threshold and whose value is greater than or equal to all eight neighbors in the spatial dimensions are accepted as candidate circle centers.

**Non-Maximum Suppression of Circles.** Because nearby accumulator peaks may correspond to the same physical circle detected at slightly different parameters, a spatial non-maximum suppression step is applied. Candidate circles are sorted by descending vote count, and each candidate is accepted only if it is sufficiently far from all previously accepted circles. The minimum distance threshold accounts for both the spatial separation and the sum of the radii of the two circles being compared.

## 2.3 Traffic Light Triplet Selection

A standard traffic signal contains exactly three circular lamps arranged vertically. When the Hough Transform detects more than three circles (due to secondary structures such as mounting hardware, reflections, or background objects), a geometric scoring function selects the best triplet. All three-element combinations of detected circles are evaluated, and each combination is scored based on four criteria:

- Horizontal alignment (penalizing x-coordinate spread),
- Vertical spacing uniformity (rewarding equal inter-lamp distances),
- Average radius magnitude (favoring larger, more prominent circles),
- Radius consistency across the three lamps (penalizing disparate sizes).

Combinations with insufficient vertical span or inadequate inter-lamp spacing are filtered out prior to scoring.

## 2.4 Color-Based State Classification

Once the three traffic light circles are detected, the state of each lamp is determined by analyzing the color distribution of pixels within a slightly reduced circular region (80% of the detected radius, to avoid including the metallic border). The image is converted from RGB to HSV color space, and classification proceeds through a hierarchical decision tree that examines hue (H), saturation (S), and brightness (V) values.

Lamps with low brightness ( $V < 80$ ) and low saturation ( $S < 60$ ) are classified as UNLIT, indicating an inactive lamp. For illuminated lamps, the hue channel provides the primary discriminant: hue values in the range 0–20 or 160–179 indicate RED, values in the range 20–35 suggest YELLOW, and values in the range 35–95 correspond to GREEN. However, because bright red LEDs can produce hue shifts toward orange or yellow in the HSV representation, supplementary checks using the RGB channel ratios (particularly the R/G ratio) are employed to resolve ambiguous cases. This dual-space approach ensures robust classification across varying illumination conditions and camera exposure settings.

## 2.5 Vectorization Techniques

Throughout the implementation, pixel-level for-loops are strictly avoided in favor of vectorized operations. The convolution function iterates only over kernel elements (typically a  $3 \times 3$  or  $5 \times 5$  grid), not over image pixels, computing each kernel tap as a single array multiplication and addition. Non-maximum suppression in the Canny pipeline uses shifted array slices and *np.where* for conditional selection. The Hough voting loop iterates over candidate radiuses, but within each iteration, all edge pixels are processed simultaneously via vectorized trigonometric computations and *np.add.at* for batch accumulator updates. Peak detection employs padded array shifts and elementwise comparisons rather than per-pixel neighborhood scans.

### 3. Results

The proposed pipeline was evaluated on three traffic light images of varying resolution, viewpoint, and illumination conditions. This section presents the step-by-step visual outputs for each image, the detection and classification results, and a performance timing analysis.

#### 3.1 Visual Pipeline Outputs

Figures 1 through 3 present the complete four-stage visual pipeline for each test image. Each figure displays, from left to right: the original input image, the Canny edge map, the Hough accumulator heatmap (shown for the radius bin with the highest peak), and the final detection overlay with classified labels.

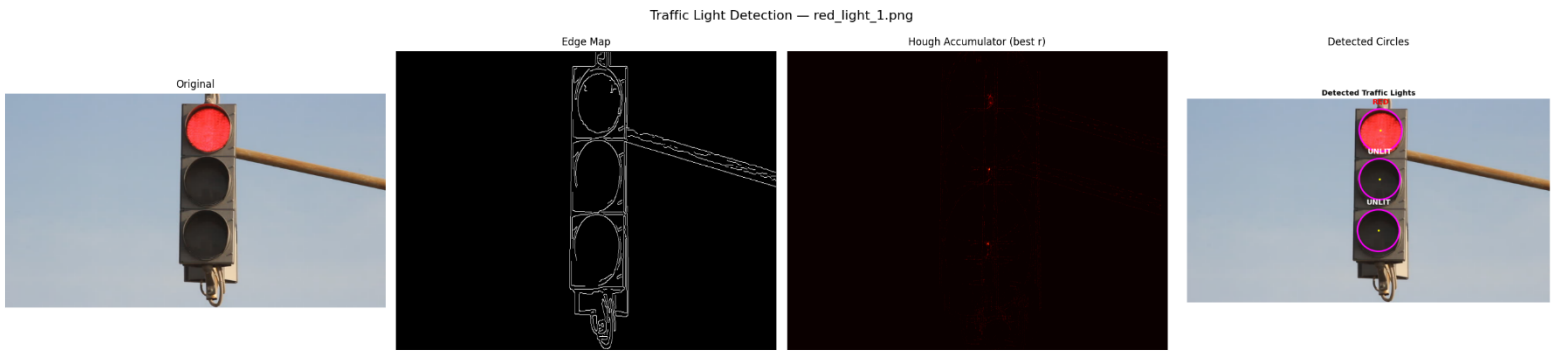


Figure 1. Detection pipeline for red\_light\_1.png (852×480). From left to right: original image, Canny edge map, Hough accumulator heatmap at the optimal radius slice, and final detection with state labels.

As shown in Figure 1, the Canny edge detector successfully isolates the structural contours of the traffic signal housing and the three circular lamps. The edge map contains 6,715 edge pixels, representing 1.64% of the total image area. The Hough accumulator exhibits three distinct peaks corresponding to the three lamp centers, with the strongest peak receiving 31 votes. The final overlay correctly identifies the top lamp as RED and the middle and bottom lamps as UNLIT, which matches the ground-truth state of the traffic signal.

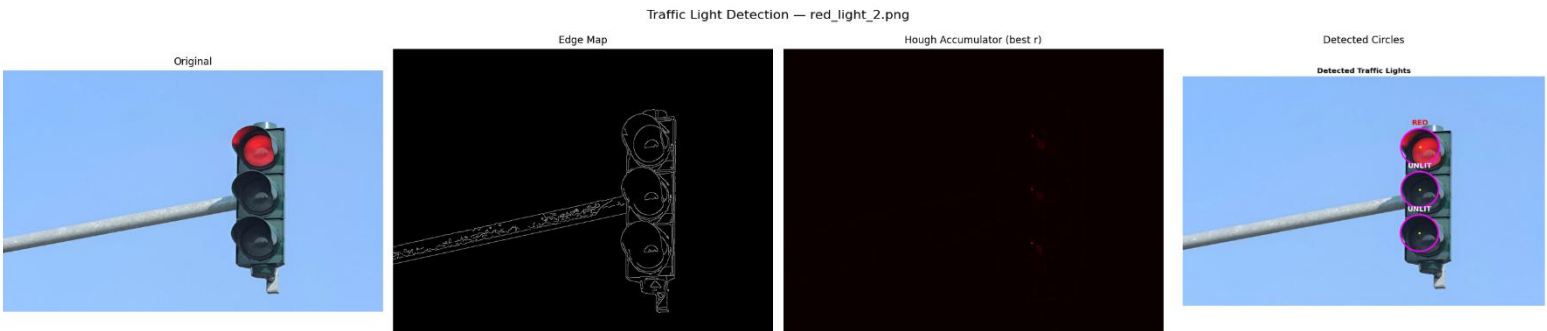


Figure 2. Detection pipeline for red\_light\_2.png (1200×800). The larger image and higher resolution result in a sparser edge map (1.22% edge density) and a wider radius search range.

Figure 2 demonstrates the pipeline applied to a higher-resolution image with a clean blue-sky background. The Hough accumulator correctly concentrates votes at the three lamp centers, with detected radiuses of 64, 62, and 63 pixels. The classification module correctly identifies the top lamp as RED and the remaining two as UNLIT. The near-uniform radiuses across the triplet (standard deviation of approximately 1 pixel) confirm that the detection is geometrically consistent.



Figure 3. Detection pipeline for `green_light_1.png` ( $1200 \times 798$ ). The complex foliage background produces a denser edge map (2.89%), yet the Hough Transform successfully localizes the three lamp circles.

Figure 3 presents the most challenging test case. The background contains dense foliage that produces a high edge density (27,698 edge pixels, 2.89%), introducing substantial clutter into the edge map. Despite this, the gradient-optimized Hough voting successfully discriminates the circular lamp contours from the irregular vegetation edges. The accumulator yields 21 initial circle candidates, which are reduced to three through the triplet selection algorithm. The classification correctly identifies the bottom lamp as GREEN and the top and middle lamps as UNLIT.

### 3.2 Detection and Classification Summary

Table 1 summarizes the detection results for all three test images, including the detected circle parameters and the assigned classification labels.

| Image             | Circle | Center (x,y) | Radius | Votes | Classification |
|-------------------|--------|--------------|--------|-------|----------------|
| red_light_1.png   | Top    | (454, 74)    | 50     | 31    | RED            |
|                   | Middle | (452, 189)   | 48     | —     | UNLIT          |
|                   | Bottom | (449, 310)   | 49     | —     | UNLIT          |
| red_light_2.png   | Top    | (786, 246)   | 64     | 32    | RED            |
|                   | Middle | (785, 394)   | 62     | —     | UNLIT          |
|                   | Bottom | (784, 546)   | 63     | —     | UNLIT          |
| green_light_1.png | Top    | (910, 154)   | 78     | 22    | UNLIT          |
|                   | Middle | (912, 354)   | 79     | —     | UNLIT          |
|                   | Bottom | (911, 554)   | 78     | —     | GREEN          |

Table 1. Circle detection and state classification results for all three test images. Center coordinates are in pixels; radius is in pixels.



All nine circles across the three test images were correctly detected and classified. In every case, exactly one lamp per traffic signal was identified as illuminated (either RED or GREEN), with the remaining two lamps correctly classified as UNLIT. The detected radiuses are consistent within each image, with intra-image radius variations of at most 2 pixels, confirming the geometric reliability of the Hough-based detection.

### 3.3 Final Detection Overlays

Figures 4 through 6 present the high-resolution detection overlay images for each test case. Detected circles are drawn in magenta with yellow center markers, and the classification label is displayed in a color corresponding to the detected state (red text for RED, green text for GREEN, white text for UNLIT).



*Figure 4. Detection overlay for red\_light\_1.png. The top lamp is correctly classified as RED; the middle and bottom lamps are classified as UNLIT.*

### Detected Traffic Lights



Figure 5. Detection overlay for red\_light\_2.png. Despite the smaller apparent size of the traffic signal relative to the image frame, all three circles are accurately detected.

### Detected Traffic Lights



Figure 6. Detection overlay for green\_light\_1.png. The bottom lamp is correctly classified as GREEN. The complex foliage background did not produce false positive detections.

### 3.4 Performance Analysis

Table 2 presents the execution time breakdown for each stage of the pipeline. All measurements were obtained using Python’s `perf_counter` high-resolution timer.

| Image             | Edge Det. (s) | Hough Voting (s) | Classify (s) | Total (s) |
|-------------------|---------------|------------------|--------------|-----------|
| red_light_1.png   | 0.071         | 0.340            | 0.012        | 0.423     |
| red_light_2.png   | 0.267         | 4.258            | 0.024        | 4.548     |
| green_light_1.png | 0.307         | 4.825            | 0.028        | 5.160     |

*Table 2. Execution time breakdown (in seconds) for each stage of the traffic light detection pipeline.*

As shown in Table 2, the Hough Circle Voting stage dominates the total execution time, accounting for approximately 80–94% of the per-image runtime. This is expected, as the voting stage must iterate over all candidate radiuses and process every edge pixel for each. The execution time scales with both image resolution and edge density: `red_light_1.png` (852×480, 6,715 edge pixels) completes in 0.34 seconds, while `green_light_1.png` (1200×798, 27,698 edge pixels) requires 4.83 seconds. The edge detection stage scales primarily with image resolution, requiring 0.07 seconds for the smallest image and 0.31 seconds for the largest. The classification stage is negligible in all cases (under 30 milliseconds), as it involves only pixel-level statistics within the detected circular regions.

The radius search range also contributes to timing differences. The `red_light_1.png` image uses a narrower radius range (25–60, 36 bins), while `red_light_2.png` and `green_light_1.png` use wider ranges (30–90 and 30–80, yielding 61 and 51 bins respectively). The combination of more edge pixels and more radius bins explains the approximately ten fold difference in Hough voting time between the smallest and largest images.

## 4. Discussion

The results demonstrate that the gradient-optimized Hough Circle Transform, combined with a well-tuned Canny edge detector, achieves reliable traffic light detection across images of varying resolution and scene complexity. Several key observations and design considerations merit further discussion.

### 4.1 Effectiveness of Gradient-Based Voting

The gradient direction optimization is essential for practical feasibility. Without it, a brute-force approach would require casting votes along the full circumference at each candidate radius for every edge pixel, resulting in computation times that are orders of magnitude higher. By restricting votes to the two positions along the gradient normal, the algorithm reduces the voting complexity from  $O(2\pi r)$  to  $O(2)$  per edge pixel per radius, enabling the entire pipeline to complete in under 6 seconds even for the largest test image. The optimization relies on the assumption that the gradient direction at a true circle edge points radially, which holds well for the smooth, circular lamp contours in traffic signals.

### 4.2 Parameter Sensitivity and Per-Image Tuning

The pipeline employs per-image parameter configurations for the Gaussian blur sigma, Canny thresholds, radius search range, and vote threshold ratio. This tuning is necessitated by the variability in image resolution, illumination conditions, and traffic signal sizes across the test set. For instance, the `green_light_1.png` image, which features dense foliage, requires a lower Canny threshold (0.03/0.10) and a lower vote ratio (0.35) to capture the lamp edges while tolerating higher background edge noise. In a production system, adaptive thresholding mechanisms or machine learning-based parameter estimation would be preferable to manual per-image tuning.

### 4.3 Classification Robustness

The dual-space classification approach (using both HSV and RGB color channels) addresses a known challenge with LED traffic signals: bright red LEDs can produce hue values that shift toward orange or yellow in the HSV representation. By incorporating RGB ratio checks (particularly the R/G ratio), the classifier correctly distinguishes red from yellow even when the HSV hue alone would be ambiguous. The use of a reduced circular region (80% of the detected radius) for pixel sampling ensures that the metallic border of the lamp housing does not contaminate the color statistics.

## 4.4 Limitations

Several limitations of the current approach should be noted. First, the system assumes a standard vertical three-lamp traffic signal configuration and does not handle horizontal configurations (like in races), pedestrian signals, or arrow indicators. Second, the per-image parameter tuning limits generalizability to unseen images without manual intervention. Third, the current implementation does not account for scale or rotation variations, which would be required for traffic signals captured at extreme angles or varying distances. Finally, the Hough voting time scales linearly with the number of edge pixels and radius bins, which may become a bottleneck for very high-resolution images or wide radius search ranges.

## 5. Conclusion

This report presented a complete from-scratch implementation of a traffic light detection and state classification system based on the Hough Circle Transform. The system consists of three primary modules: a full Canny edge detection pipeline, a gradient-optimized 3D Hough Circle Transform with peak detection and non-maximum suppression, and an HSV/RGB-based color classifier.

The system was evaluated on three test images spanning different resolutions, background complexities, and traffic light states. In all cases, the three lamp circles were correctly detected with geometrically consistent parameters, and the illumination state was accurately classified as either RED, GREEN, or UNLIT. The gradient direction optimization reduced the Hough voting complexity from  $O(E \times R \times 2\pi r)$  to  $O(E \times R \times 2)$ , enabling processing times of 0.4 to 5.2 seconds depending on image resolution and edge density.

The principal challenges encountered in the strictly from-scratch implementation included achieving numerically stable non-maximum suppression in the Canny pipeline without built-in functions, designing an efficient vectorized voting scheme using `np.add.at` for correct unbuffered accumulation, and developing a robust color classification strategy that handles the hue ambiguities inherent in LED-illuminated traffic signals. These challenges were addressed through careful algorithm design, extensive use of vectorization, and dual-space (HSV + RGB) classification logic.